GRAPHIOT : discours de presentation

Version orale, francaise, decontractee

Billal Mokhtari

Octobre 2026

Note : un bloc par slide. Le ton est volontairement relax, en tutoiement. Les equations sont expliquees a l'oral, sans jargon inutile.

Slide 1. Page de titre

Salut. Alors aujourd'hui je te presente le papier qu'on vise pour le numero special de l'IEEE Internet of Things Journal, celui sur le graph learning pour l'IoT. L'idee en une phrase : des agents LLM commencent a piloter des usines, et on veut detecter quand un de ces agents envoie une commande dangereuse, meme quand elle a l'air parfaitement normale. Je te fais le tour du probleme, de notre solution, de comment on compte le tester, et je garde deux slides a la fin pour te montrer ce qu'il reste a corriger dans l'article.

Slide 2. Le contexte

Le point de depart, c'est que maintenant on branche des agents LLM sur des usines. L'agent ne touche jamais les machines directement. Il passe par une passerelle, un gateway, qui lui offre quelques outils. Quand l'agent appelle un outil, la passerelle traduit ca en une ecriture Modbus vers un automate, un PLC.

Le souci, c'est que l'agent travaille avec les droits d'un operateur de confiance. Donc si on le trompe, par exemple avec une injection de prompt, ou s'il se trompe tout seul, il peut envoyer une commande qui est valide, autorisee, et meme dans la liste blanche. Techniquement rien n'est illegal. Et pourtant ca peut casser un lot. C'est ca le coeur du probleme.

Slide 3. La commande piege : context-shifted

La pour bien faire comprendre, je prends une image toute bete. "Rouler a 90", c'est nickel sur l'autoroute, et c'est dangereux devant une ecole. Meme commande, c'est juste la situation qui change.

Dans l'usine c'est pareil. Ecrire une vitesse d'enduction de 2,40, en regime stable le secheur est chaud, le film seche, le lot est bon. Mais pendant la chauffe du secheur, quand le four est encore froid, la meme commande fait sortir le film tout mouille, et le lot est foutu. Nous on appelle ca une commande "context-shifted", une commande decalee de son contexte. Elle est normale dans un contexte, nuisible dans un autre.

Slide 4. Pourquoi les defenses actuelles sont aveugles

Et la le probleme, c'est qu'aucune defense actuelle ne voit ca, parce que chacune ne regarde qu'un seul cote.

Le garde d'agent voit l'outil, le tag, la sequence d'appels, mais il ne connait pas la phase du procede. Le detecteur reseau voit le paquet Modbus et les flux, mais il ne connait ni la tache ni la phase. Le moniteur de procede voit les capteurs, mais avant que l'effet arrive il ne voit rien d'anormal, et de toute facon il ne sait pas qui a envoye la commande.

C'est ce qu'on formalise avec l'Observation 1. Si ton score ne depend de la commande que par son triplet, l'outil, le tag et le bin de la valeur, plus l'historique de l'agent, alors la commande benigne et son jumeau dangereux recoivent exactement le meme score. C'est pas une opinion, c'est mecanique : si deux commandes ont les memes entrees, elles ont le meme score. Donc c'est perdu d'avance pour une vue unique.

Slide 5. La definition formelle

La je pose les definitions proprement. Une commande, c'est un quadruplet $c = (a, \kappa, v, t)$: l'outil a, le tag cible κ, la valeur v et le temps t.

La fonction $\mathrm{bin}(v)$ range la valeur dans un des Q bins quantiles, qu'on apprend sur les donnees benignes. En clair, au lieu de garder la valeur exacte, on dit juste dans quelle tranche elle tombe. Et $\mathcal{C}_{\text{benign}}$, c'est juste l'ensemble de tous les triplets qu'on a deja vus en fonctionnement normal.

Du coup une commande est context-shifted si les trois conditions tiennent en meme temps : son triplet est deja connu, donc elle a l'air normale ; la liste blanche l'autorise, donc elle passe la passerelle ; et l'arbitre dit qu'elle est nuisible. C'est exactement la tranche de test qu'on va rapporter a part, parce que c'est la que tout se joue.

Slide 6. Notre idee : un seul graphe a la passerelle

Notre idee, c'est d'arreter de regarder un seul cote. On construit un graphe dynamique et heterogene a la passerelle, a partir des logs et du bus de terrain. Ce graphe relie la tache, l'agent, la passerelle, les machines du reseau, les automates, les tags et les etapes de production.

Le truc important, c'est que ce graphe connecte la tache et l'etat de l'usine, ce qu'aucune vue unique ne fait. Ensuite un GNN temporel apprend, uniquement sur du fonctionnement normal, a quel point chaque commande est probable dans son contexte. Et on note la commande avant que son effet apparaisse. Sur le dessin, les deux noeuds en couleur, la Tache et l'Etape, ce sont justement ceux que les detecteurs classiques ne relient jamais.

Slide 7. Le schema du graphe

La je te montre vite fait de quoi est fait le graphe, juste pour que tu voies que c'est concret. On a sept types de noeuds : la tache, l'agent, la passerelle, les hotes du reseau, les automates, les tags et les etapes. Chacun avec ses features, par exemple un tag porte sa valeur courante et sa moyenne glissante, une etape porte sa phase et depuis combien de temps elle y est.

Et il y a plusieurs types d'aretes : qui a assigne la tache, quel outil est appele, les ecritures et lectures avec le code de fonction et la valeur, les flux reseau, et les liens statiques genre quel automate controle

quel tag.

La regle a retenir, et je vais y revenir tout le temps, c'est qu'aucune feature ne contient de recette, de limite de securite, ou de sortie du jumeau numerique. Le detecteur ne voit jamais ces choses la. C'est ce qui rend le test honnete. Les fenetres font 10 secondes, avec un pas de 5 secondes.

Slide 8. Le modele, du graphe a la probabilite

Cote modele, le pipeline est assez standard, et c'est voulu. D'abord chaque type de noeud est projete en dimension 64. Ensuite deux couches de HGT, des transformers de graphe heterogenes, melangent l'information entre voisins, en tenant compte du type d'arete. Puis un GRU par noeud melange l'information dans le temps.

L'equation du GRU, c'est juste ca :

$$\mathbf{m}_v^t = \mathrm{GRU}\big(\mathbf{h}_v^t,\ \mathbf{m}_v^{t-1}\big)\,.$$

Ca veut dire : la nouvelle memoire du noeud v a l'instant t, c'est un melange de ce qu'on vient de calculer $\mathbf{h}_v^t$, et de l'ancienne memoire $\mathbf{m}_v^{t-1}$. En gros chaque noeud se souvient de son passe recent. Donc l'etape "secheur" peut encoder "ca fait 40 secondes que je suis en chauffe", pas juste la derniere seconde. Un nouveau noeud, lui, demarre avec une memoire a zero. Pour resumer : le HGT c'est l'espace, le GRU c'est le temps.

Slide 9. La vraisemblance conditionnee a la tache et le score

Et la on arrive au coeur. Le modele est entraine uniquement sur du normal, et il doit predire le tag ecrit et le bin de la valeur, a partir du contexte juste avant la commande. C'est l'equation :

$$p_\theta\Big(\kappa, \mathrm{bin}(v)\ \Big|\ \mathbf{m}_\tau^{t^-}, \mathbf{m}_{\mathrm{agent}}^{t^-}, \mathbf{m}_{\mathrm{stage}(\kappa)}^{t^-}, \mathbf{m}_\kappa^{t^-}\Big)\,.$$

Je te la lis en francais : c'est la probabilite que l'agent ecrive ce tag κ avec une valeur dans ce bin, sachant la memoire de la tache, celle de l'agent, celle de l'etape du tag, et celle du tag lui-meme. Le petit t^-, c'est le dernier instantane avant la commande. Donc le modele ne voit jamais l'effet, il predit avant.

Et le score d'anomalie, c'est juste la surprise :

$$s(c) = -\log p_\theta(\kappa, \mathrm{bin}(v) \mid \cdot)\,.$$

Si la commande est tres probable, le log est proche de zero, donc petite surprise. Si elle est tres improbable, moins le log d'un petit nombre ca fait un grand nombre, donc grosse surprise, donc alarme. Et le point malin : une commande context-shifted a un triplet frequent, donc en apparence banale, mais un contexte improbable. Sa surprise ne sera grande que si le modele utilise vraiment la tache et l'etape. C'est exactement ce qu'on veut tester. Le seuil d'alarme est regle sur les donnees benignes pour avoir 1 pour cent de faux positifs.

Slide 10. Comment on le teste honnetement

Maintenant, le protocole, parce que c'est la moitie de la valeur du papier. On veut un test qui peut echouer, pas une demo qui marche a tous les coups.

Les attaques sont generees par un agent red-team, un LLM d'une autre famille que l'operateur. Il recoit juste le texte d'une technique MITRE ATT&CK pour l'ICS et la doc de l'usine. On n'ecrit aucun script d'attaque nous-memes, comme ca on ne triche pas en sa faveur.

Les labels viennent d'un arbitre externe, qui regarde les sorties du procede et les compare aux limites de securite. Et le detecteur ne voit jamais ces limites. Ensuite on se compare a six baselines, de la simple table de valeurs par phase jusqu'a un juge LLM, en passant par la liste blanche, un modele tabulaire, une vue reseau et une vue agent. Et les splits sont chronologiques, jamais aleatoires, et les seuils ne voient jamais les donnees d'attaque.

Slide 11. L'attaquant adaptatif

Le test le plus dur, c'est l'attaquant adaptatif. La on suppose un attaquant en grey-box, qui connait l'architecture et peut interroger notre score. Il resout ce petit probleme :

$$\min_{\delta \in \mathcal{R}_B(e)} \max_{c \in e \oplus \delta} s_\theta(c) \quad \text{tel que} \quad \text{Oracle}(e \oplus \delta) = \text{nuisible}.$$

En mots : il cherche la modification δ qui fait baisser le plus possible notre plus gros score sur l'episode, mais avec une contrainte en beton, l'arbitre doit toujours dire que c'est nuisible. Donc il n'a pas le droit de desactiver l'attaque pour passer inapercu. Les modifications autorisees sont realistes : inserer des lectures benignes, remettre d'anciennes commandes normales, decouper un changement, decaler dans le temps. Et il a un budget de B evenements en plus, avec B qui vaut 0, 4 ou 8. Ca nous empeche de clamer une robustesse qu'on n'a pas.

Slide 12. Les hypotheses pre-enregistrees

Avant de lancer quoi que ce soit, on fige trois hypotheses, et elles ont le droit d'echouer. H1, la principale : sur la tranche context-shifted, notre rappel a 1 pour cent de faux positifs bat les baselines d'au moins 10 points. H2 : les versions mono-vue restent sous 0,6 d'AUROC, et enlever le noeud tache fait plus mal qu'enlever le reseau. H3 : avec un budget de 4, moins d'episodes nuisibles nous echappent qu'aux deux baselines fortes.

Et surtout, si H1 echoue contre la table par phase, ca veut dire que la tache n'apporte rien de plus que les stats de phase, et on l'ecrit quand meme. C'est ca l'honnetete du protocole.

Slide 13. A corriger dans l'article, 1 sur 2

La je passe aux deux slides qui t'interessent sans doute le plus, ce qu'il faut encore corriger.

D'abord un souci de notation. Le symbole τ est utilise pour deux choses differentes : la tache, dans la section methode, et la marge de l'arbitre, dans la section evaluation, quand on dit "depasse ses limites d'au moins τ". Un relecteur va tiquer. Je propose de renommer la marge, par exemple en ε.

Ensuite, le planning est deja depasse. La decision sur l'arbitre de secours etait datee du 17 octobre, l'enregistrement OSF du 22, et le pilote du 25. On est apres. Il faut qu'on refixe des dates realistes ensemble, sachant que la deadline de soumission est le 30 novembre.

Slide 14. A corriger dans l'article, 2 sur 2

Cote contenu, il reste plusieurs decisions. Tous les placeholders a remplir : la phrase de resultat dans l'abstract, tous les crochets dans les tableaux, les noms et tailles des modeles pour l'operateur et le red-team, et les heures de capture.

Il faut aussi figer les hyperparametres finaux avec la grille de recherche, le temps d'entrainement et le nombre de parametres. Verrouiller le choix de l'arbitre, le primaire ou celui de secours, avec la table des limites en annexe. L'hypothese H2, le "AUROC sous 0,6", est assez forte, donc soit on confirme qu'elle tient au pilote, soit on l'adoucit. La figure 1, l'architecture, est encore une boite vide, il faut produire le vrai PDF vectoriel. Et enfin verifier que la formulation "emule contre synthetique" est coherente avec le papier dataset.

Slide 15. Ou on en est et la suite

Pour conclure. Notre contribution tient en trois points : une classe nommee et mesurable, les commandes context-shifted ; une vraisemblance conditionnee a la tache, calculee a la passerelle et entrainee seulement sur du normal ; et un test qui peut echouer, avec red-team independant, arbitre externe et attaquant adaptatif.

La suite immediate : on verrouille l'arbitre et les dates, on lance le pilote avec le test go/no-go, puis la campagne principale, et on remplit les tableaux et la figure finale. Voila, je suis preneur de tes retours, surtout sur le planning et sur ce qu'on garde ou pas.